The framework provides a binder specifically for implementing integration tests focusing on the business logic rather than the middleware. Let's see how it works, using Dispatcher Service as an example.

NOTE The test binder provided by Spring Cloud Stream is meant to verify the correct configuration and integration with a technology-agnostic destination binder. If you want to test the application against a specific broker (in our case, it would be for RabbitMQ), you can rely on Testcontainers, as you learned in the previous chapter. I'll leave that up to you as an exercise.

First, add a dependency to the test binder in the build.gradle file of your Dispatcher Service project. Unlike the other dependencies we've been working on so far, the test binder requires a more elaborate syntax to be included. For more information, refer to the Spring Cloud Stream documentation (https://spring.io/projects/spring-cloud-stream). Remember to refresh or reimport the Gradle dependencies after the new addition.

Listing 10.15 Adding a dependency for the test binder in Dispatcher Service

```
dependencies {
  ...
  testImplementation("org.springframework.cloud:spring-cloud-stream") {
    artifact {
      name = "spring-cloud-stream"
      extension = "jar"
      type ="test-jar"
      classifier = "test-binder"
    }
  }
}
```

Next, create a new `FunctionsStreamIntegrationTests` class for testing. The test setup consists of three steps:

1. Import the `TestChannelBinderConfiguration` class providing configuration for the test binder.